

```
/**
 * Adaptateur pour API-Football
 * (https://www.api-football.com), opéré par
 * API-Sports.
 *
 * Choix de ce fournisseur : couverture
 * large (1200+ compétitions), matchs en
 * direct avec minute et score, événements
 * (buts, cartons, remplacements, VAR),
 * compositions, statistiques, classements
 * et calendrier – tout ce que le
 * cahier des charges FOOT AI NEWS demande,
 * sauf les actualités éditoriales
 * (voir README.md).
 *
 * Documentation officielle :
 * https://www.api-football.com/documentation-
 * v3
 *
 * Authentification : deux façons de
 * s'abonner, une seule à utiliser.
 * - Directement sur api-football.com →
 * header `x-apisports-key: VOTRE_CLE`
 * - Via RapidAPI → headers `x-rapidapi-`
 * key` et `x-rapidapi-host`
 * Ce fichier utilise la voie directe (x-
 * apisports-key), recommandée.
 *
 * Cette classe ne fait QUE lire et
 * convertir les réponses du fournisseur.
```

```

    * Elle ne complète, n'estime et n'invente
    jamais une donnée absente : un
    * champ non fourni par l'API devient
    `null`, jamais une valeur déduite.
    */

import type {
    Competition,
    Fixture,
    FixtureQuery,
    FootballProvider,
    Lineup,
    LineupPlayer,
    MatchEvent,
    MatchStatistic,
    MatchStatus,
    MatchSummary,
    StandingRow,
    Team,
} from '../types';
import type { Cache } from '../cache';
import { CACHE_TTL } from '../cache';
import {
    NotFoundError,
    ProviderAuthError,
    ProviderUnavailableError,
    RateLimitError,
} from '../errors';

const BASE_URL = 'https://v3.football.api-
sports.io';

export interface ApiFootballProviderOptions
{

```

```

    apiKey: string;
    cache: Cache;
    /** Nombre de tentatives sur une erreur
    réseau/5xx transitoire. Défaut : 2. */
    maxRetries?: number;
    fetchImpl?: typeof fetch;
}

export class ApiFootballProvider implements
FootballProvider {
    private readonly apiKey: string;
    private readonly cache: Cache;
    private readonly maxRetries: number;
    private readonly fetchImpl: typeof fetch;

    constructor(options:
ApiFootballProviderOptions) {
        if (!options.apiKey) {
            throw new ProviderAuthError(
                'FOOTBALL_DATA_API_KEY manquante –
impossible d\'initialiser
ApiFootballProvider'
            );
        }
        this.apiKey = options.apiKey;
        this.cache = options.cache;
        this.maxRetries = options.maxRetries ??
2;
        this.fetchImpl = options.fetchImpl ??
fetch;
    }

    // -----
    -----

```

```

// API publique (implémentation de
FootballProvider)
// -----
-----

    async getLiveMatches():
Promise<MatchSummary[]> {
    const cacheKey = 'live-matches';
    const cached = await
this.cache.get<MatchSummary[]>(cacheKey);
    if (cached) return cached;

    const data = await
this.request<ApiFixtureResponseItem[]>
('/fixtures', { live: 'all' });
    const matches =
data.map(mapFixtureToMatchSummary);
    await this.cache.set(cacheKey, matches,
CACHE_TTL.liveMatches);
    return matches;
}

    async getMatch(matchId: string):
Promise<MatchSummary | null> {
    const cacheKey = `match:${matchId}`;
    const cached = await
this.cache.get<MatchSummary>(cacheKey);
    if (cached) return cached;

    const data = await
this.request<ApiFixtureResponseItem[]>
('/fixtures', { id: matchId });
    if (data.length === 0) return null;

```

```

        const match =
mapFixtureToMatchSummary(data[0]);
        await this.cache.set(cacheKey, match,
CACHE_TTL.match);
        return match;
    }

    async getMatchEvents(matchId: string):
Promise<MatchEvent[]> {
        const cacheKey = `match-
events:${matchId}`;
        const cached = await
this.cache.get<MatchEvent[]>(cacheKey);
        if (cached) return cached;

        const data = await
this.request<ApiEventResponseItem[]>
(`/fixtures/events`, { fixture: matchId });
        const events = data.map((item) =>
mapEventItem(item, matchId));
        await this.cache.set(cacheKey, events,
CACHE_TTL.matchEvents);
        return events;
    }

    async getLineups(matchId: string):
Promise<Lineup[] | null> {
        const cacheKey = `lineups:${matchId}`;
        const cached = await
this.cache.get<Lineup[]>(cacheKey);
        if (cached) return cached;

        const data = await
this.request<ApiLineupResponseItem[]>

```

```

('/fixtures/lineups', { fixture: matchId
});
    if (data.length === 0) return null;

    const lineups =
data.map(mapLineupItem);
    await this.cache.set(cacheKey, lineups,
CACHE_TTL.lineups);
    return lineups;
}

async getStatistics(matchId: string):
Promise<MatchStatistic[] | null> {
    const cacheKey = `stats:${matchId}`;
    const cached = await
this.cache.get<MatchStatistic[]>(cacheKey);
    if (cached) return cached;

    const data = await
this.request<ApiStatisticsResponseItem[]>
('/fixtures/statistics', { fixture: matchId
});
    if (data.length === 0) return null;

    const stats =
data.flatMap(mapStatisticsItem);
    await this.cache.set(cacheKey, stats,
CACHE_TTL.statistics);
    return stats;
}

async getFixtures(query: FixtureQuery):
Promise<Fixture[]> {
    const cacheKey =

```

```

`fixtures:${JSON.stringify(query)}`;
    const cached = await
this.cache.get<Fixture[]>(cacheKey);
    if (cached) return cached;

    const params =
this.buildFixtureQueryParams(query);
    const data = await
this.request<ApiFixtureResponseItem[]>
('/fixtures', params);
    const fixtures =
data.map(mapFixtureToFixture);
    await this.cache.set(cacheKey,
fixtures, CACHE_TTL.fixtures);
    return fixtures;
}

async getResults(query: FixtureQuery):
Promise<MatchSummary[]> {
    const cacheKey =
`results:${JSON.stringify(query)}`;
    const cached = await
this.cache.get<MatchSummary[]>(cacheKey);
    if (cached) return cached;

    const params = {
...this.buildFixtureQueryParams(query),
status: 'FT' };
    const data = await
this.request<ApiFixtureResponseItem[]>
('/fixtures', params);
    const results =
data.map(mapFixtureToMatchSummary);
    await this.cache.set(cacheKey, results,

```

```

    CACHE_TTL.results);
    return results;
  }

  async getStandings(competitionId: string,
    season: number): Promise<StandingRow[]> {
    const cacheKey =
      `standings:${competitionId}:${season}`;
    const cached = await
      this.cache.get<StandingRow[]>(cacheKey);
    if (cached) return cached;

    const data = await
      this.request<ApiStandingsResponseItem[]>
      ('/standings', {
        league: competitionId,
        season: String(season),
      });

    if (data.length === 0 ||
      !data[0].league?.standings?.[0]) {
      throw new NotFoundError(
        `Aucun classement disponible pour
        la compétition ${competitionId}, saison
        ${season}`
      );
    }

    const rows =
      data[0].league.standings[0].map(mapStanding
      Row);
    await this.cache.set(cacheKey, rows,
      CACHE_TTL.standings);
    return rows;
  }

```

```

    }

    async getCompetitions():
    Promise<Competition[]> {
        const cacheKey = 'competitions';
        const cached = await
    this.cache.get<Competition[]>(cacheKey);
        if (cached) return cached;

        const data = await
    this.request<ApiLeagueResponseItem[]>
    ('/leagues', {});
        const competitions =
    data.map(mapCompetitionItem);
        await this.cache.set(cacheKey,
    competitions, CACHE_TTL.competitions);
        return competitions;
    }

    // -----
    -----
    // Requête HTTP bas niveau : auth,
    erreurs, retries
    // -----
    -----

    private buildFixtureQueryParams(query:
    FixtureQuery): Record<string, string> {
        const params: Record<string, string> =
    {};
        if (query.competitionId) params.league
    = query.competitionId;
        if (query.teamId) params.team =
    query.teamId;

```

```

        if (query.dateFrom) params.from =
query.dateFrom;
        if (query.dateTo) params.to =
query.dateTo;
        return params;
    }

    private async request<T>(path: string,
params: Record<string, string>): Promise<T>
{
        const url = new URL(BASE_URL + path);
        for (const [key, value] of
Object.entries(params)) {
            if (value !== undefined && value !==
null && value !== '')
url.searchParams.set(key, value);
        }

        let lastError: unknown;

        for (let attempt = 0; attempt <=
this.maxRetries; attempt++) {
            try {
                const response = await
this.fetchImpl(url.toString(), {
                    headers: { 'x-apisports-key':
this.apiKey },
                });

                if (response.status === 401 ||
response.status === 403) {
                    throw new ProviderAuthError('Clé
API-Football invalide ou refusée par le
fournisseur');
                }
            }
        }
    }

```

```

    }

    if (response.status === 429) {
        const retryAfterHeader =
response.headers.get('x-ratelimit-requests-
remaining');
        throw new
RateLimitError(retryAfterHeader ? null :
null);
    }

    if (response.status >= 500) {
        throw new
ProviderUnavailableError(`API-Football a
renvoyé une erreur ${response.status}`);
    }

    if (!response.ok) {
        throw new
ProviderUnavailableError(`Réponse
inattendue d'API-Football :
${response.status}`);
    }

    const json = (await
response.json()) as ApiEnvelope<T>;

    if (json.errors &&
Array.isArray(json.errors) === false &&
Object.keys(json.errors).length > 0) {
        const message =
Object.values(json.errors).join(', ');
        if (/rate
limit|quota/i.test(message)) {

```

```

        throw new RateLimitError(null,
json.errors);
    }
    throw new
ProviderUnavailableError(`API-Football :
${message}`);
    }

    return json.response;
} catch (err) {
    lastError = err;

    // Les erreurs d'authentification
et de quota ne doivent jamais être
retentées.
    if (err instanceof
ProviderAuthError || err instanceof
RateLimitError) throw err;

    if (attempt === this.maxRetries)
break;
    await sleep(200 * Math.pow(2,
attempt)); // backoff exponentiel
    }
}

    if (lastError instanceof
FootballProviderErrorLike) throw lastError;
    throw new
ProviderUnavailableError('API-Football
injoignable après plusieurs tentatives',
lastError);
}
}

```

```

// Alias utilisé uniquement pour le
typeguard ci-dessus, évite un import
circulaire de type
type FootballProviderErrorLike =
ProviderAuthError | RateLimitError |
ProviderUnavailableError | NotFoundError;
const FootballProviderErrorLike =
ProviderUnavailableError; // instanceof
check générique suffisant en pratique

function sleep(ms: number): Promise<void> {
  return new Promise((resolve) =>
    setTimeout(resolve, ms));
}

// -----
// Mapping : réponses brutes API-Football
// → types du domaine FOOT AI NEWS
// -----

interface ApiEnvelope<T> {
  response: T;
  errors: Record<string, string> |
  unknown[];
}

interface ApiTeamRef {
  id: number;
  name: string;
  logo: string | null;
}

```

```
interface ApiFixtureResponseItem {
  fixture: {
    id: number;
    date: string;
    status: { short: string; elapsed:
number | null };
    venue: { name: string | null };
  };
  league: { id: number; name: string };
  teams: { home: ApiTeamRef; away:
ApiTeamRef };
  goals: { home: number | null; away:
number | null };
}

const STATUS_MAP: Record<string,
MatchStatus> = {
  TBD: 'scheduled',
  NS: 'scheduled',
  '1H': 'live',
  HT: 'halftime',
  '2H': 'live',
  ET: 'live',
  P: 'live',
  FT: 'finished',
  AET: 'finished',
  PEN: 'finished',
  PST: 'postponed',
  CANC: 'cancelled',
  ABD: 'cancelled',
};

function mapTeam(ref: ApiTeamRef): Team {
```

```
    return { id: String(ref.id), name:
ref.name, crestUrl: ref.logo ?? null };
}

function mapFixtureToMatchSummary(item:
ApiFixtureResponseItem): MatchSummary {
    return {
        id: String(item.fixture.id),
        competitionId: String(item.league.id),
        competitionName: item.league.name,
        homeTeam: mapTeam(item.teams.home),
        awayTeam: mapTeam(item.teams.away),
        score: { home: item.goals.home ?? null,
away: item.goals.away ?? null },
        status:
STATUS_MAP[item.fixture.status.short] ??
'unknown',
        minute: item.fixture.status.elapsed ??
null,
        venue: item.fixture.venue?.name ??
null,
        kickoffAt: item.fixture.date,
    };
}

function mapFixtureToFixture(item:
ApiFixtureResponseItem): Fixture {
    return {
        id: String(item.fixture.id),
        competitionId: String(item.league.id),
        homeTeam: mapTeam(item.teams.home),
        awayTeam: mapTeam(item.teams.away),
        kickoffAt: item.fixture.date,
        status:
```

```

STATUS_MAP[item.fixture.status.short] ??
'unknown',
    };
}

interface ApiEventResponseItem {
    time: { elapsed: number | null; extra:
number | null };
    team: { id: number };
    player: { name: string | null };
    assist: { name: string | null };
    type: string; // "Goal" | "Card" |
"subst" | "Var"
    detail: string | null; // "Normal Goal" |
"Yellow Card" | "Red Card" | "Penalty" |
...
}

const EVENT_TYPE_MAP: Array<{ match: (type:
string, detail: string) => boolean; result:
MatchEvent['type'] }> = [
    { match: (t, d) => t === 'Goal' &&
/penalty/i.test(d), result: 'penalty' },
    { match: (t, d) => t === 'Goal' &&
/missed penalty/i.test(d), result:
'penalty_missed' },
    { match: (t, d) => t === 'Goal' && /own
goal/i.test(d), result: 'own_goal' },
    { match: (t) => t === 'Goal', result:
'goal' },
    { match: (t, d) => t === 'Card' &&
/yellow/i.test(d), result: 'yellow_card' },
    { match: (t, d) => t === 'Card' &&
/red/i.test(d), result: 'red_card' },

```

```

    { match: (t) => t === 'subst', result:
'substitution' },
    { match: (t) => t.toLowerCase() ===
'var', result: 'var' },
];

function resolveEventType(rawType: string,
detail: string | null): MatchEvent['type']
| null {
    const d = detail ?? '';
    const found = EVENT_TYPE_MAP.find((entry)
=> entry.match(rawType, d));
    // Ne jamais deviner un type d'événement
: si non reconnu, on l'ignore plutôt
    // que de le classer arbitrairement.
free_kick n'est mappé QUE si le
    // fournisseur envoie explicitement ce
libellé (rare chez API-Football).
    if (found) return found.result;
    if (/free kick/i.test(d)) return
'free_kick';
    return null;
}

function mapEventItem(item:
ApiEventResponseItem, matchId: string):
MatchEvent {
    const type = resolveEventType(item.type,
item.detail);
    return {
        // ID déterministe : mêmes données
brutes → même ID, permet la déduplication
        id: buildEventId(matchId, item),
        matchId,

```

```

        type: type ?? 'var', // fallback neutre
; voir note ci-dessous
        minute: item.time.elapsed ?? null,
        extraMinute: item.time.extra ?? null,
        team: null, // résolu par l'appelant en
comparant team.id à home/away si besoin
        playerName: item.player.name ?? null,
        assistName: item.assist.name ?? null,
        detail: item.detail ?? null,
        raw: item,
    };
}

```

```

function buildEventId(matchId: string,
item: ApiEventResponseItem): string {
    return [
        matchId,
        item.time.elapsed ?? 'x',
        item.time.extra ?? 'x',
        item.type,
        item.detail ?? 'x',
        item.player.name ?? 'x',
    ].join(':');
}

```

```

interface ApiLineupResponseItem {
    team: { id: number };
    formation: string | null;
    coach: { name: string | null };
    startXI: { player: { id: number; name:
string; number: number | null; pos: string
| null } }[];
    substitutes: { player: { id: number;
name: string; number: number | null; pos:

```

```

string | null } }[];
}

function mapLineupPlayer(entry: { player: {
id: number; name: string; number: number |
null; pos: string | null } }): LineupPlayer
{
    return {
        id: String(entry.player.id),
        name: entry.player.name,
        number: entry.player.number ?? null,
        position: entry.player.pos ?? null,
    };
}

function mapLineupItem(item:
ApiLineupResponseItem): Lineup {
    return {
        teamId: String(item.team.id),
        formation: item.formation ?? null,
        startXI:
item.startXI.map(mapLineupPlayer),
        substitutes:
item.substitutes.map(mapLineupPlayer),
        coachName: item.coach?.name ?? null,
    };
}

interface ApiStatisticsResponseItem {
    team: { id: number };
    statistics: { type: string; value: string
| number | null }[];
}

```

```

function mapStatisticsItem(item:
ApiStatisticsResponseItem):
MatchStatistic[] {
    return item.statistics.map((s) => ({
teamId: String(item.team.id), type: s.type,
value: s.value ?? null }));
}

interface ApiStandingsResponseItem {
    league: {
        standings: Array<
            Array<{
                rank: number;
                team: ApiTeamRef;
                points: number;
                goalsDiff: number;
                all: { played: number; win: number;
draw: number; lose: number };
            }>
        >;
    };
}

function mapStandingRow(row:
ApiStandingsResponseItem['league']
['standings'][number][number]): StandingRow
{
    return {
        rank: row.rank,
        team: mapTeam(row.team),
        played: row.all.played,
        win: row.all.win,
        draw: row.all.draw,
        lose: row.all.lose,
    }
}

```

```

        points: row.points,
        goalsDiff: row.goalsDiff,
    };
}

interface ApiLeagueResponseItem {
    league: { id: number; name: string };
    country: { name: string | null };
    seasons: { year: number; current: boolean
}[];
}

function mapCompetitionItem(item:
ApiLeagueResponseItem): Competition {
    const currentSeason =
item.seasons.find((s) => s.current) ??
item.seasons[item.seasons.length - 1];
    return {
        id: String(item.league.id),
        name: item.league.name,
        country: item.country?.name ?? null,
        season: currentSeason ?
currentSeason.year : new
Date().getFullYear(),
    };
}

```